

Coral Shop — Análisis del código actual frente a la arquitectura y al objetivo del negocio

Grupo 1 · Desarrollo Web Integrado (UTP) · 7 de octubre de 2026

Actualización (7 de octubre de 2026). Tras este análisis el equipo decidió **eliminar por completo la integración con CJ Dropshipping**: la tienda no obtiene productos de otros comercios; todos los productos base a personalizar se cargan y configuran desde el panel admin de Coral Shop (decisión D8 de `fases.md`). Las menciones a CJ que siguen abajo describen el código tal como estaba al momento del análisis.

Fuentes comparadas:

1. `clase_referencias/Coral_Shop_Arquitectura_Actual_y_Evolucion_v1.docx` (v1.1, octubre 2026).
2. Material de clase (`imagen_01` a `imagen_03`): los cuatro estilos de arquitectura (capas, cliente–servidor, MVC, dirigida por eventos), la dinámica «*el restaurante en capas*» (Controller → Service → Repository, «*¡el mozo NO entra a la despensa!*») y la *Mesa de Partes Digital* (Controller + DTO valida formato → 400; Service aplica reglas → 422; Repository persiste; «*cada capa conoce SOLO a la de abajo*»).
3. El código real del repositorio: `backend/` (Spring Boot 3.5, Java 21) y `frontend/` (React 19 + Vite), revisado archivo por archivo.
4. El objetivo del negocio definido por el equipo (ver §1).

0. Resumen ejecutivo

Veredicto. Coral Shop hoy es un **catálogo administrado con carrito provisional**. La base técnica es sana (autenticación con sesión + CSRF, roles, Flyway, validaciones, transacciones), pero **el núcleo del negocio no existe todavía**: no hay personalización (estampado/bordado), ni venta por mayor, ni checkout, ni pedidos, ni seguimiento de entrega. El documento Word describe el estado del código con bastante fidelidad (con dos imprecisiones, §2), pero su hoja de ruta está orientada a un *dropshipping con CJ*, que **no es** el modelo de negocio de Coral Shop.

Dimensión	Estado	Comentario
Arquitectura cliente–servidor	✅ Alineado	React (cliente) ↔ API REST Spring (servidor) ↔ PostgreSQL.
MVC (adaptado a SPA)	✅ Alineado	React = Vista, <code>@RestController</code> = Controlador, entidades/registros = Modelo.
Arquitectura en capas	⚠️ Parcial	<code>auth</code> cumple Controller → Service → Repository; todo <code>catalog</code> mete SQL en los controllers (el « <i>mozo entra a la despensa</i> »).
Dirigida por eventos	🚫 No aplica aún	No hay eventos (correcto según el Word); se propone uno con consumidor real en §7 (fase 4).

Dimensión	Estado	Comentario
Flujo de compra del negocio	✗ Ausente	Falta personalización, mayoreo, checkout, pedido, pago y entrega.
Coherencia con el informe del entregable 1	✗ Divergente	El informe declara Servlets + JDBC + JWT y 28 tablas; el código es Spring Boot + sesión y 13 tablas.

Camino recomendado (detalle en §7): mantener Spring Boot, ordenar las capas como en clase, y construir **el flujo vertical completo** *producto* → *personalizar* → *cantidad* → *carrito* → *checkout* → *pedido* → *seguimiento hasta la entrega* antes que cualquier otra cosa. Eliminar la integración con CJ Dropshipping (ya ejecutado, ver D8).

1. El norte del proyecto

Coral Shop es una tienda en línea de prendas y accesorios **personalizables**:

1. El cliente recorre una **lista de productos** (polos, poleras, gorras, tote bags...).
2. Elige un producto y lo **personaliza con estampado o bordado**: sube **su imagen**, escoge una o varias **zonas predefinidas según el tipo de producto** (p. ej. un polo: pecho, espalda, manga; una gorra: frente, lateral) y ve una **vista previa**.
3. Decide la **cantidad**: **por unidad** o **por mayor** (3, 6, 12 o múltiplos de 12).
4. Continúa el flujo de **carrito** → **checkout (datos de entrega)** → **pago** → **confirmación**.
5. Sigue el pedido **hasta la entrega** (estados de producción y despacho).

Este es el criterio con el que se evalúa todo lo que sigue.

2. ¿El Word describe bien el código?

Afirmación del Word	Verificación en el código	Estado
Frontend React 19 + Vite 8 + Tailwind; backend Spring Boot 3.5, Java 21, MVC, Security, validación, JPA y JDBC; PostgreSQL + Flyway	<code>frontend/package.json</code> , <code>backend/pom.xml</code> , migraciones V1 y V2	✓ Correcto
Registro, login y logout con sesión y CSRF; roles USER y ADMIN	<code>SecurityConfig</code> , <code>AuthController</code> , <code>RegistrationService</code> ; el frontend pide <code>GET /api/auth/csrf</code> antes de cada POST	✓ Correcto
Catálogo, detalle y variantes leídos de la API	<code>GET /api/products</code> y <code>GET /api/products/{id}</code> (<code>CatalogController</code>)	✓ Correcto
«El carrito se conserva al recargar la misma pestaña»	El carrito vive solo en un <code>useReducer</code> (<code>features/cart/model/cart-provider.jsx</code>); se pierde al recargar . No usa <code>sessionStorage</code> ni <code>localStorage</code>	✗ Impreciso
Las reglas de creación/importación están en los controllers	<code>AdminProductController</code> , <code>CjImportController</code> , <code>CatalogController</code> y <code>AdminOverviewController</code> usan <code>JdbcTemplate</code> directamente	✓ Correcto

Afirmación del Word	Verificación en el código	Estado
Pantallas admin de usuarios, pedidos y categorías llaman a rutas no implementadas	El frontend llama <code>GET /api/users</code> , <code>PUT /api/users/{id}/role</code> , <code>GET /api/orders</code> , <code>PUT /api/orders/{id}/status</code> , <code>POST/PUT/DELETE /api/categories/{id}</code> ; ninguna existe en el backend	✓ Correcto
No existen eventos, bus ni broker	Confirmado	✓ Correcto
Pendiente: checkout, pagos, pedidos, favoritos, edición/eliminación de productos	Confirmado	✓ Correcto
(No lo menciona)	El backend no tiene pruebas (<code>backend/src/test</code> no existe)	⚠ Omisión
(No lo menciona)	El frontend tiene ~34 archivos muertos (no alcanzables desde <code>main.jsx</code>), algunos llaman a <code>fakestoreapi.com</code>	⚠ Omisión

Conclusión: como fotografía del presente, el Word es fiable. Su debilidad está en el **futuro que propone** (§5): gira alrededor de CJ Dropshipping y de un carrito de productos terminados, sin mencionar la personalización ni el mayoreo, que son la razón de ser de la tienda.

3. Alineación con la arquitectura vista en clase

3.1 Arquitectura en capas (restaurante / Mesa de Partes)

La regla de clase: **Controller** (valida formato, DTO, 400) → **Service** (reglas de negocio, 422, transacciones) → **Repository** (datos). *Cada capa conoce solo a la de abajo.*

Componente	Controller	Service	Repository	¿Cumple?
Registro (auth)	<code>AuthController</code> + <code>RegisterRequest</code> (DTO con <code>@Valid</code>)	<code>RegistrationService</code> (normaliza, valida, <code>BCrypt</code> , <code>@Transactional</code>)	<code>UserRepository</code> , <code>RoleRepository</code> (JPA)	✓ Ejemplo perfecto del patrón de clase
Catálogo público	<code>CatalogController</code>	—	SQL embebido en el controller	✗ El mozo entra a la despensa
Alta de producto	<code>AdminProductController</code> + <code>CreateProductRequest</code>	— (reglas y SQL en el controller)	SQL embebido	✗
Importación CJ	<code>CjImportController</code>	—	SQL embebido + <code>CjClient</code>	✗
Métricas admin	<code>AdminOverviewController</code>	—	SQL embebido	✗

Otras observaciones de capas:

- **Paquetes por funcionalidad** (`auth`, `catalog`, `user`): es correcto y el Word lo recomienda; dentro de cada uno conviene separar `controller` / `service` / `repository` / `dto` para que la

rúbrica vea las capas sin ambigüedad.

- **Códigos de error:** se usa `400` y `409`; las violaciones de reglas de negocio deberían responder `422` (como en la Mesa de Partes) desde el Service.
- **Mensajes mezclados:** `RegistrationService` responde en español, `AdminProductController` en inglés («*Select an active category*»). Unificar en español.

3.2 Cliente-servidor —

Separación limpia: el navegador solo presenta y recoge datos; permisos y persistencia están en el servidor. Pendiente importante: **el servidor todavía no recalcula precios** porque no hay pedidos; cuando existan, el precio enviado por el navegador **nunca** debe usarse (el Word lo advierte con razón).

3.3 MVC — (adaptado)

React cumple el papel de Vista y Spring el de Controlador, devolviendo JSON. Es la misma adaptación que el informe del entregable 1 ya justifica (§2.2.1: «la Vista JSP se externaliza a React»).

3.4 Dirigida por eventos —

No existe y el Word acierta al no inventarla. Pero el profesor la presentó como uno de los cuatro estilos, así que conviene **incorporar un caso real y pequeño** cuando exista el pedido (§7, fase 4).

4. Alineación con el objetivo del negocio (el flujo completo)

#	Paso del negocio	Frontend	Backend	Base de datos	Estado
1	Lista de productos	<code>products-page.jsx</code> llama a <code>GET /api/products</code> . Filtro por categoría y orden por precio en el navegador ; categorías fijas en el código (Hombre/Mujer/Unisex); el buscador del header no hace nada	<code>GET /api/products</code> sin paginación ni filtros	<code>products</code> , <code>categories</code> , <code>product_images</code>	 Básico
2	Tipo de producto (polo, polera, gorra, tote)	No existe; solo categorías por género	No existe	No hay <code>product_type</code>	
3	Talla y color	<code><select></code> de variantes con stock por variante	Variantes en <code>GET /api/products/{id}</code>	<code>product_variants</code> , <code>sizes</code> , <code>colors</code>	
4	Técnica: estampado o bordado	No existe	No existe	No existe	

#	Paso del negocio	Frontend	Backend	Base de datos	Estado
5	Subir la imagen del cliente	No existe (ningún <code>input type=file</code> ; el admin solo pega una URL)	No hay endpoint <i>multipart</i>	No existe	✗
6	Zonas predefinidas según el tipo de producto	No existe	No existe	No existe	✗
7	Vista previa y recargo por personalización	No existe	No existe	No existe	✗
8	Cantidad por unidad o por mayor (3, 6, 12, 12·k)	Máximo 5 unidades por línea (<code>MAX_CART_ITEMS = 5</code>), incrementos de 1	No existe	No existe	✗ Contradice el negocio
9	Carrito	En memoria; se pierde al recargar; barra de envío gratis fija en \$50	No existe	No existe	⚠ Provisional
10	Checkout (dirección, envío)	Botón «Proceed to checkout» sin acción	No existe	<code>orders</code> tiene campos de dirección (sin distritos de Perú)	✗
11	Pago	No existe	No existe	No existe	✗
12	Confirmación del pedido	No existe	No existe	<code>orders</code> , <code>order_items</code> (sin personalización)	✗
13	Seguimiento hasta la entrega	No existe (la cuenta solo muestra usuario y correo)	No existe	<code>orders.status</code> : PENDING / SHIPPED / DELIVERED / CANCELLED (falta producción)	✗
14	Gestión de pedidos (admin)	Pantalla lista, llama a <code>/api/orders</code>	No existe	<code>orders</code>	⚠ Solo UI
15	Alta de productos (admin)	Formulario completo con variantes	POST <code>/api/admin/products</code>	—	✅ (sin editar/eliminar)

Tensiones con el modelo de negocio:

- **CJ Dropshipping vs. personalización.** CJ vende productos terminados que se envían desde el proveedor; la personalización exige **prendas en blanco en stock local** que se estampan o bordan en el taller. La integración CJ (≈ 25 % del backend) no aporta al flujo principal. Decisión: **eliminarla** (D8).
- **Moneda y mercado.** Los precios se muestran como \$ y la importación usa USD; la empresa es peruana: debe ser **S/ (PEN)**, con distritos/provincias de Perú en la dirección.

- **Idioma.** La interfaz está en inglés (`lang="en"`); para la exposición y el informe debe estar en español.
- **Stock y mayoreo.** Con el tope de 5 unidades no se puede vender ni una docena.

5. Comparación con el informe del entregable 1

Tema	Informe del entregable 1	Código real	Recomendación
Stack backend	Servlets + JDBC + Tomcat (Java 17)	Spring Boot 3.5 (Java 21)	Mantener Spring Boot y justificarlo: el material de clase actual (fotos) ya enseña <i>Controller / Service / Repository</i> , que es exactamente el vocabulario de Spring. Spring MVC corre sobre un <code>DispatcherServlet</code> en un Tomcat embebido: los Servlets siguen ahí, debajo.
Seguridad	JWT + BCrypt	Sesión <code>JSESSIONID</code> + CSRF + BCrypt	Mantener sesión + CSRF (más simple y segura para una SPA del mismo dominio). Actualizar el informe.
Endpoints	<code>/api/v1/...</code>	<code>/api/...</code>	Adoptar uno; lo barato es actualizar el informe a <code>/api/...</code> .
Paquete	<code>pe.edu.utp.coralshop</code>	<code>com.coralshop</code>	Cosmético; se puede renombrar en la refactorización de capas.
Modelo de datos	28 tablas (5 módulos, incluye personalización)	13 tablas (sin personalización)	El diseño del informe es la meta ; implementarlo por fases vía Flyway (§7).
Personalización	Solo estampado ; zonas globales (no por tipo de producto)	No existe	Añadir bordado como técnica y relacionar zonas con tipo de producto .
Mayoreo	No contemplado	No existe	Añadir reglas de cantidad y escalas de precio (§7, D2).
Pruebas	Declara TDD con JUnit + Mockito	Cero pruebas	Escribir al menos las pruebas de las reglas de precio y cantidad (son lógica pura, ideales para TDD).

6. Calidad y deuda técnica

Frontend

- ~34 archivos muertos: `app.jsx`, `router/router.jsx`, `app/home|login|products`, todo `common/`, todo `components/`, `pages/home.jsx`, `features/login/`, `features/cart/context/`, `cartdropdown.jsx`, `features/products/component/`, `features/products/pages/`, `hooks use-get-products*` y servicios que llaman a `fakestoreapi.com`. Generan confusión al exponer el código.
- Protección de rutas dentro de los componentes (sin guardas a nivel de router): funciona, pero el backend es el que realmente protege (correcto).
- Errores detectados: `useOrders.updateStatus` sin `try/catch`; el formulario de contacto y el de suscripción no tienen manejador; `categoryName.toLowerCase()` sin protección ante nulos; la

tarjeta *Unisex* reutiliza la imagen de mujer; IDs de rol fijos (1/2) en la pantalla de usuarios.

Backend

- Controllers con SQL (§3.1).
- Rutas autorizadas en `SecurityConfig` que no tienen controller (`/api/users/**`, `/api/orders/**`, `/api/categories/**` para escritura).
- Sin pruebas, sin paginación, sin manejo global de errores (`@RestControllerAdvice`).
- Despliegue: sesión y CSRF con frontend en Vercel y backend en Railway requieren que `/api` se sirva por el mismo dominio (el `vercel.json` ya hace *rewrite*; verificar el login en producción).

7. Mejor camino a seguir

7.1 Decisiones que hay que cerrar primero

#	Decisión	Propuesta
D1	Stack backend	Spring Boot (se queda). El informe del entregable 2 lo justifica con el material de capas de clase.
D2	Regla de cantidad	Modalidad UNIDAD : 1 a 2 piezas a precio unitario. Modalidad MAYOR : paquetes de 3, 6, 12 o múltiplos de 12 , con descuento escalonado. <i>Confirmar si «por unidad» permite 2 piezas o solo 1.</i>
D3	Composición del paquete	Un paquete comparte el mismo diseño y técnica , pero puede repartirse entre tallas y colores (p. ej. 12 polos: 4 S, 4 M, 4 L). Es lo realista para pedidos de empresas y promociones.
D4	Técnicas	Estampado (DTF/serigrafía) y bordado . El bordado tiene zonas y tamaños más limitados y un costo base mayor.
D5	Zonas	Predefinidas por tipo de producto : polo y polera (pecho izquierdo, pecho centro, espalda, manga), gorra (frente, lateral), tote (cara A, cara B). Cada zona tiene medidas máximas y recargo.
D6	Pago	Simulado (tarjeta de prueba / Yape simulado) con estados reales; no integrar una pasarela real para el curso.
D7	Archivos subidos	PNG/JPG ≤ 5 MB. Guardarlos en PostgreSQL (<code>bytea</code>) o en un volumen de Railway; el disco de Railway es efímero.
D8	Origen de los productos	Sin proveedores externos : todos los productos base se cargan desde el panel admin. La integración CJ se elimina .
D9	Idioma y moneda	Español y S/ .

7.2 Fases

Fase 0 — Ordenar la casa (1 a 2 días)

- Borrar el código muerto del frontend; traducir la interfaz al español; precios en S/.
- Persistir el carrito en `localStorage` (y quitar el tope de 5).
- Refactorizar `catalog` a `controller` → `service` → `repository` (como `auth`), con un `@RestControllerAdvice` que traduzca errores a 400 / 404 / 409 / 422.

Fase 1 — Modelo de datos del negocio (migración Flyway v3)

Partiendo de `scripts/esquema.py` (para que informe y código no se desincronicen):

Tabla	Propósito
<code>product_type</code>	POLO, POLERA, GORRA, TOTE; <code>products.product_type_id</code>
<code>customization_technique</code>	ESTAMPADO, BORDADO: costo base y restricciones
<code>print_zone</code> + <code>product_type_zone</code>	Zonas por tipo de producto, medidas máximas y recargo por técnica
<code>quantity_tier</code>	Escalas de precio: 1, 3, 6, 12 (y 12·k) con su factor de descuento
<code>design_upload</code>	Imagen subida por el cliente (archivo, tipo, tamaño, dueño)
<code>order_item_customization</code>	Técnica, zonas y diseño de cada línea del pedido
<code>address</code> , <code>order_status_history</code> , <code>payment</code>	Entrega en Perú, trazabilidad del estado y pago simulado

Estados del pedido: `PENDIENTE_PAGO` → `PAGADO` → `EN_PRODUCCION` → `LISTO` → `ENVIADO` → `ENTREGADO` (y `CANCELADO`).

Fase 2 — API del flujo de compra (backend)

Endpoint	Capa de negocio
<code>GET /api/products/{id}/customization-options</code>	Técnicas y zonas válidas para el tipo de producto
<code>POST /api/designs (multipart)</code>	Valida formato y peso; devuelve el id del diseño
<code>POST /api/quotes</code>	<code>PricingService</code> : recalcula precio base + recargos + escala de mayoreo
<code>POST /api/orders</code>	<code>OrderService</code> en una transacción : valida la regla de cantidad (D2), recalcula el precio, descuenta stock con actualización condicional y crea el pedido y sus líneas
<code>GET /api/orders/me</code> , <code>GET /api/orders/me/{id}</code>	Historial y seguimiento del cliente
<code>POST /api/orders/{id}/payment</code>	Pago simulado → <code>PAGADO</code>
<code>GET /api/orders</code> , <code>PUT /api/orders/{id}/status</code>	Admin: las mismas rutas que el frontend ya llama , con máquina de estados
<code>GET /api/users</code> , <code>PUT /api/users/{id}/role</code> , <code>CRUD /api/categories</code>	Completa las pantallas admin existentes

Fase 3 — Frontend del flujo

1. **Personalizador** en pasos: producto → color y talla → técnica → zona(s) → subir imagen → **vista previa** (imagen superpuesta sobre la prenda con `<canvas>` o CSS) → modalidad y cantidad (con reparto por tallas en mayoreo) → precio recalculado por el servidor → agregar al carrito.
2. **Carrito** con líneas personalizadas (miniatura del diseño, zonas, técnica, paquete).
3. **Checkout**: dirección (departamento/provincia/distrito), método de envío, pago simulado.
4. **Confirmación** con código de pedido y **Mis pedidos** con línea de tiempo de estados.
5. **Admin**: bandeja de pedidos con descarga del archivo del diseño y cambio de estado.

Fase 4 — Evento real (cubre el cuarto estilo de arquitectura)

`OrderStatusChanged`, publicado por `OrderService` tras el `commit` (`@TransactionalEventListener`), con un consumidor que registra una **notificación** para el cliente («Tu pedido pasó a EN_PRODUCCION») y otro que escribe la **auditoría**. Es pequeño, real y demostrable en la exposición, sin broker.

Fase 5 — Calidad, demo y documentos

- Pruebas JUnit de `PricingService` y de la regla de cantidad (TDD real), y una prueba de integración de `POST /api/orders`.
- Datos de demostración (productos por tipo, zonas, técnicas, escalas) y colección Postman.
- Actualizar `esquema.py` → figuras ER, informe y PPT del entregable 2.

7.3 Prioridad según la rúbrica

Criterio (pts)	Qué lo mueve más
Implementación (6)	Fases 2 y 3: flujo completo de punta a punta, con validaciones e integridad
Diseño de la solución (4)	Fase 1 + historias de usuario del personalizador y del mayoreo, con diagramas actualizados
Organización del informe (3)	Alinear el informe con el código real (§5)
Exposición (4)	Una demo que recorra el flujo del §1 sin cortes
Diagnóstico estratégico (3)	Ya cubierto en el entregable 1; solo reflejar el mayoreo en Canvas y objetivos

En una frase: primero una sola historia de punta a punta —*un cliente personaliza 12 polos con su logo en el pecho, paga y ve su pedido pasar de PAGADO a ENTREGADO*—; todo lo demás (favoritos, banners, cupones) espera.